23. Design a Load Balancer

Key Terms:

Term	Full Name	What it does
L4	Layer 4 (Transport)	Routes based on IP/port, doesn't inspect content
L7	Layer 7 (Application)	Routes based on HTTP headers, URL, cookies
VIP	Virtual IP	Single IP address representing multiple servers
VRRP	Virtual Router Redundancy Protocol	Failover protocol for LB high availability
Anycast	Network routing	Same IP announced from multiple locations
SSL Termination	TLS decryption	LB decrypts HTTPS, forwards HTTP to backends
Health Check	Backend monitoring	Periodic probes to detect unhealthy servers

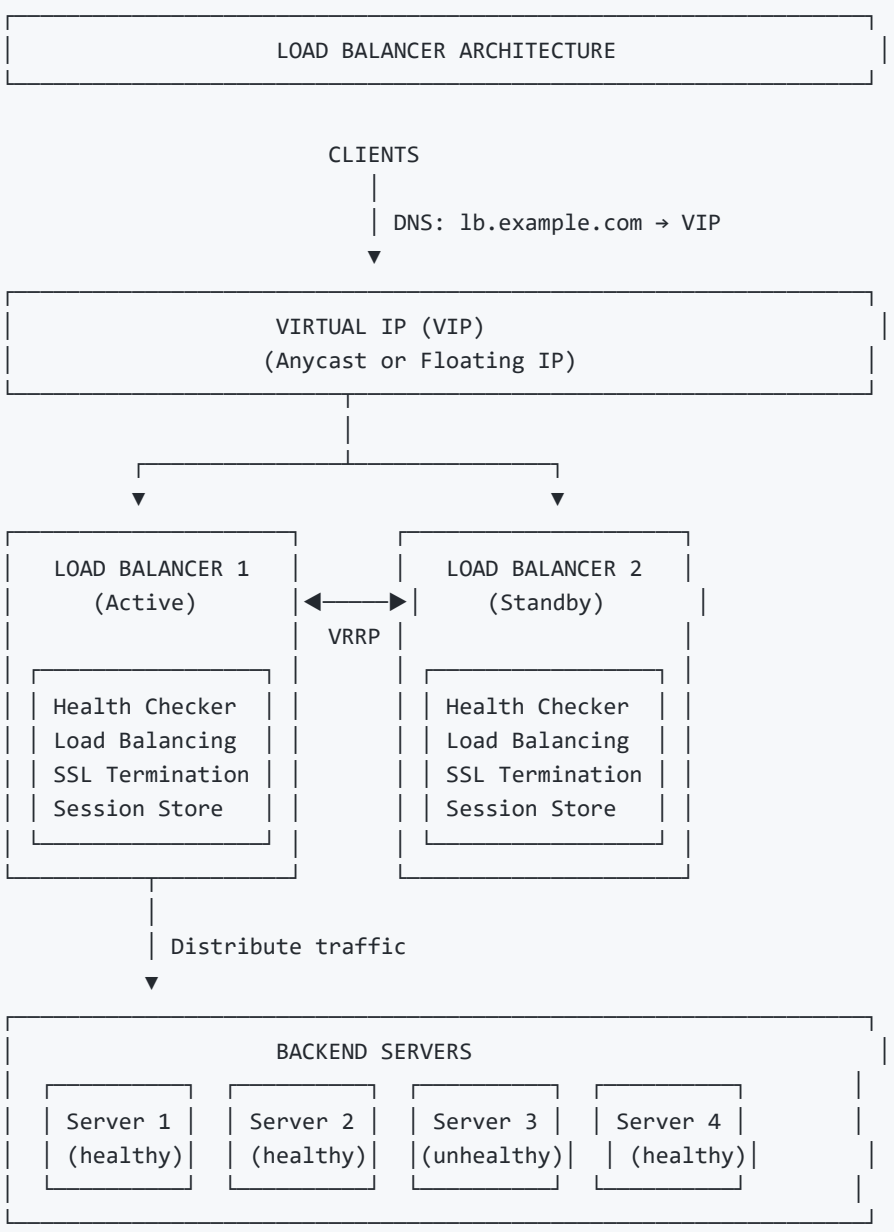
Problem Statement

Design a load balancer that distributes traffic across multiple backend servers with health checking, session persistence, and high availability.

Clarifying Questions to Ask

1. **Layer**: L4 (TCP) or L7 (HTTP)?
2. **Scale**: How many requests per second? Backend servers?
3. **Features**: SSL termination, session stickiness, rate limiting?
4. **Health checks**: Active probing or passive monitoring?
5. **Algorithm**: Round-robin, least connections, weighted?
6. **Availability**: Active-passive or active-active LB?
7. **Protocol**: HTTP, gRPC, WebSocket, raw TCP?

High-Level Design



API Design

Load Balancer Configuration APIs

Method	Endpoint	Description
`POST`	`/api/v1/pools`	Create a backend pool
`PUT`	`/api/v1/pools/{poolId}`	Update pool configuration
`POST`	`/api/v1/pools/{poolId}/targets`	Add backend target

`DELETE`	`/api/v1/pools/{poolId}/targets/{targetId}`	Remove backend target

Health Check APIs

Method	Endpoint	Description
`GET`	`/api/v1/pools/{poolId}/health`	Get health status of all backends
`PUT`	`/api/v1/pools/{poolId}/healthcheck`	Configure health check rules

Routing Rules APIs

Method	Endpoint	Description
`POST`	`/api/v1/rules`	Create routing rule (path/header-based)
`GET`	`/api/v1/rules`	List routing rules
`PUT`	`/api/v1/rules/{ruleId}`	Update routing rule

Monitoring APIs

Method	Endpoint	Description
`GET`	`/api/v1/stats`	Get request distribution stats
`GET`	`/api/v1/stats/latency?backend={id}`	Get per-backend latency
`GET`	`/api/v1/stats/connections`	Get active connection count

Load Balancing Algorithms

```
1. ROUND ROBIN
┌──────────────────────────────────────────────────────────────┐
│  Request 1 → Server A                                        │
│  Request 2 → Server B                                        │
│  Request 3 → Server C                                        │
│  Request 4 → Server A (cycle repeats)                        │
│                                                              │
│  Pros: Simple, even distribution                             │
│  Cons: Ignores server load/capacity                          │
└──────────────────────────────────────────────────────────────┘

2. WEIGHTED ROUND ROBIN
┌──────────────────────────────────────────────────────────────┐
│  Server A: weight=3, Server B: weight=1                      │
│                                                              │
```

```
  Request 1 → A
  Request 2 → A
  Request 3 → A
  Request 4 → B

  Use case: Heterogeneous servers (different capacities)
```

```
3. LEAST CONNECTIONS
```

```
  Server A: 10 active connections
  Server B: 5 active connections
  Server C: 8 active connections

  New request → Server B (least connections)

  Use case: Long-lived connections, varying request duration
```

```
4. IP HASH (Consistent Hashing)
```

```
  hash(client_ip) % num_servers = server_index

  Client 1.2.3.4 → always Server A
  Client 5.6.7.8 → always Server B

  Use case: Session stickiness without cookies
```

```
5. LEAST RESPONSE TIME
```

```
  Track average response time per server
  Route to server with lowest response time

  Use case: Optimize for latency
```

Health Checking

```
ACTIVE HEALTH CHECKS:
```

```
  LB periodically probes each backend:

  GET /health HTTP/1.1
  Host: backend-server

  Healthy response:
    HTTP 200 OK
    {"status": "healthy", "db": "ok", "cache": "ok"}

  Unhealthy thresholds:
    - 3 consecutive failures → mark unhealthy
    - 2 consecutive successes → mark healthy
```

```
│                                                                  │
│  Interval: 5-10 seconds                                          │
└──────────────────────────────────────────────────────────────────┘

PASSIVE HEALTH CHECKS:
┌──────────────────────────────────────────────────────────────────┐
│  Monitor real traffic for failures:                              │
│    - 5xx errors                                                  │
│    - Connection timeouts                                         │
│    - Connection resets                                           │
│                                                                  │
│  If error rate > threshold → mark unhealthy                      │
│  Pros: No extra traffic, catches issues active checks miss       │
└──────────────────────────────────────────────────────────────────┘
```

Session Persistence (Sticky Sessions)

```
METHODS:
┌──────────────────────────────────────────────────────────────────┐
│  1. Cookie-based:                                                │
│     Set-Cookie: SERVERID=server-a; Path=/                        │
│     LB reads cookie, routes to same server                       │
│                                                                  │
│  2. Source IP:                                                   │
│     hash(client_ip) → consistent server                          │
│     Problem: NAT, proxies share IP                               │
│                                                                  │
│  3. Application-generated session ID:                            │
│     LB inspects session_id header/cookie                         │
│     Maintains session→server mapping                             │
│                                                                  │
│  Best practice: Use external session store (Redis)               │
│  instead of sticky sessions for stateless backends               │
└──────────────────────────────────────────────────────────────────┘
```

L4 vs L7 Load Balancing

```
L4 (Transport Layer):
┌──────────────────────────────────────────────────────────────────┐
│  Routes based on: IP address, TCP port                           │
│  Cannot inspect: HTTP headers, cookies, URL path                 │
│                                                                  │
│  Pros: Fast, low overhead, protocol-agnostic                     │
│  Cons: Limited routing options                                   │
│                                                                  │
│  Use case: Database connections, raw TCP, high throughput        │
└──────────────────────────────────────────────────────────────────┘

L7 (Application Layer):
┌──────────────────────────────────────────────────────────────────┐
```

```
│  Routes based on: URL path, headers, cookies, body             │
│                                                                 │
│  /api/* → API servers                                          │
│  /static/* → CDN or static servers                             │
│  Header: X-Feature: beta → Beta servers                        │
│                                                                 │
│  Pros: Flexible routing, SSL termination, caching              │
│  Cons: Higher latency, more resource intensive                 │
│                                                                 │
│  Use case: HTTP APIs, microservices, content-based routing     │
└─────────────────────────────────────────────────────────────────┘
```

High Availability

```
ACTIVE-PASSIVE WITH VRRP:

┌─────────────────────────────────────────────────────────────────┐
│                    Virtual IP: 10.0.0.100                       │
│                           │                                     │
│              ┌────────────┴────────────┐                        │
│              ▼                         ▼                        │
│      ┌──────────────┐           ┌──────────────┐                │
│      │    LB 1      │           │    LB 2      │                │
│      │  (Active)    │◄─────────►│  (Standby)   │                │
│      │ 10.0.0.101   │ Heartbeat │ 10.0.0.102   │                │
│      └──────────────┘           └──────────────┘                │
│                                                                 │
│  If LB1 fails:                                                  │
│    1. Heartbeat timeout detected                                │
│    2. LB2 claims Virtual IP                                     │
│    3. Failover in 1-3 seconds                                   │
└─────────────────────────────────────────────────────────────────┘
```

Trade-offs

Decision	Option A	Option B	Recommendation
Layer	L4	L7	**L7** - more routing options
Algorithm	Round-robin	Least connections	**Least conn** - handles variance
Stickiness	Sticky sessions	External session	**External** - better scaling
Health check	Active	Active + Passive	**Both** - comprehensive

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference
Load Balancer	L4/L7 traffic distribution with health checking	Tech Guide
Consistent Hashing	IP hash algorithm for session stickiness	Tech Guide

How to Explain It Well

> **Opening**: "A load balancer distributes traffic for scalability and reliability. I'll design an L7 LB with health checking and HA."

> **Key talking points**:
>
> - "L7 enables content-based routing: /api to API servers, /static to CDN"
> - "Least connections algorithm handles varying request durations"
> - "Active health checks every 5 seconds, passive monitors real traffic"
> - "VRRP for HA: standby takes over VIP in 1-3 seconds"
> - "Avoid sticky sessions - use external session store (Redis)"
